

생성형 AI와 LLM 기반 AI 에이전트 개발

LLM API 연동 및 요청·응답 구조 이해



API란?

LLM API 연동 및 요청·응답 구조 이해

- **API(Application Programming Interface)**는 서로 다른 소프트웨어가 데이터를 주고받을 수 있도록 연결해주는 인터페이스.
쉽게 말해, 우리가 만드는 프로그램과 OpenAI의 LLM 사이를 연결해주는 창구 역할을 함.
- **LLM API를 사용하면:**
 - Python 코드에서 GPT 모델을 직접 호출 가능
 - 응답 형식, 모델 종류, 토큰 수 등을 세밀하게 제어 가능
 - 웹사이트를 거치지 않고 프로그램 안에서 바로 AI 응답을 받아 활용 가능

구분	ChatGPT/Gemini 웹사이트	LLM API
사용 방식	브라우저에서 직접 대화	Python 코드로 호출
활용 범위	개인 사용	서비스·에이전트 개발
응답 제어	제한적	모델, 토큰 등 세밀하게 제어 가능
비용	구독 요금	사용한 토큰 수만큼 과금

OpenAI API vs Gemini API

LLM API 연동 및 요청·응답 구조 이해

- AI 에이전트 개발에 주로 사용되는 두 가지 API 비교.

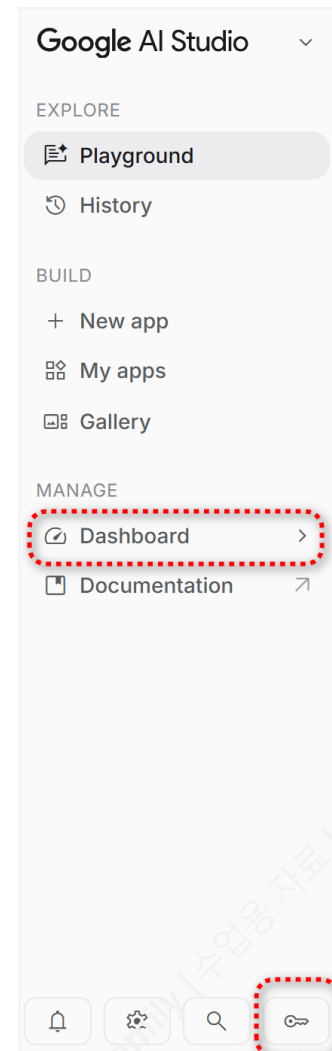
	OpenAI API	Gemini API
제공사	OpenAI	Google
대표 모델	gpt-4o-mini, gpt-4o	gemini-2.5-flash, gemini-3.1-pro
무료 티어	신규 가입 시 소액 크레딧 제공	넉넉한 무료 티어 제공
가격 (저가 모델)	저렴한 편	무료~매우 저렴
LangChain 호환성	매우 풍부	지원
레퍼런스·커뮤니티	매우 풍부 (업계 표준)	점차 확대 중
학습 용도	키 제공 후 사용	개인 키 무료 발급 가능

- 두 API 모두 Python에서 유사한 방식으로 호출할 수 있으며, 하나를 익히면 다른 것도 쉽게 적용 가능.

Gemini API Key 발급

LLM API 연동 및 요청·응답 구조 이해

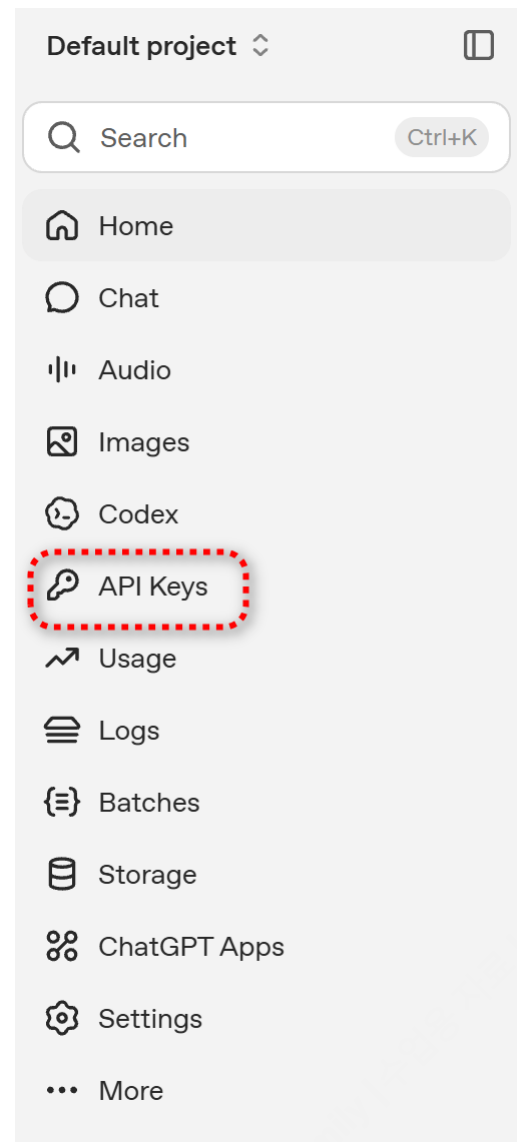
- Google AI Studio에서 무료로 발급 가능. 별도 결제 수단 등록 없이 사용할 수 있어 학습용으로 적합.
- 발급 순서:
 - ① aistudio.google.com 접속 및 Google 계정으로 로그인
 - ② 좌측 메뉴 → "Dashboard" 클릭
 - ③ "Get API key" 클릭
 - ④ "Create API key" 클릭
 - ⑤ 연결할 Google Cloud 프로젝트 선택 또는 새 프로젝트 생성
 - ⑥ 생성된 키 복사 후 안전한 곳에 저장
- API Key 관리 주의사항:
 - 발급 후 재확인 가능하지만 안전한 곳에 저장 권장
 - 코드에 직접 입력 금지 → 환경 변수(.env 파일)로 관리
 - 무료 티어 초과 시 유료 전환 필요 (수업 중 초과 가능성은 낮음)



OpenAI API Key 발급

LLM API 연동 및 요청·응답 구조 이해

- OpenAI API Key는 유료 기반.
- 발급 순서:
 - ① platform.openai.com 접속 및 로그인
 - ② 좌측 메뉴 → "API Keys" 메뉴 선택
 - ③ "Create new secret key" 클릭
 - ④ 생성된 키 복사 후 안전한 곳에 저장 (재확인 불가)



환경 변수로 API Key 관리

LLM API 연동 및 요청·응답 구조 이해

- API Key를 코드에 직접 입력하면 GitHub 등에 올릴 때 유출 위험이 있음.
환경 변수(.env 파일)로 분리해 관리하는 것이 기본 원칙.

```
# .env 파일 생성 (프로젝트 폴더 안에)
```

```
GEMINI_API_KEY=Alzaxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

```
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

```
# Python 코드에서 불러오기
```

```
from dotenv import load_dotenv
```

```
import os
```

```
load_dotenv()
```

```
gemini_key = os.getenv("GEMINI_API_KEY")
```

```
openai_key = os.getenv("OPENAI_API_KEY")
```

```
# dotenv 라이브러리 설치
```

```
pip install python-dotenv
```

- API Key는 절대 코드에 직접 입력하거나 공유하지 않도록 주의

Gemini API 요청 구조

LLM API 연동 및 요청·응답 구조 이해

- **Gemini API에 요청을 보낼 때의 기본 구조.**

```
import google.generativeai as genai
from dotenv import load_dotenv
import os

load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

model = genai.GenerativeModel(
    model_name="gemini-2.5-flash",
    system_instruction="당신은 친절한 AI 어시스턴트입니다."
)

response = model.generate_content("안녕하세요!")
print(response.text)
```

- **model_name**: 사용할 Gemini 모델 지정
- **system_instruction**: AI의 역할과 성격 설정
- **generate_content()**: 실제 요청을 보내는 메서드

Gemini API 대화 구조

LLM API 연동 및 요청·응답 구조 이해

- Gemini API에서 대화는 role과 parts로 구성된 메시지 단위로 주고받음.
role은 각 메시지가 누구의 발언인지를 구분하는 역할.

role	역할	설명
user	사용자 입력	실제 사용자가 입력하는 메시지
model	AI 응답	이전 AI 응답 (대화 맥락 유지에 활용)

- system_instruction은 별도로 설정하며, AI의 성격·역할·제약 조건을 정의함.
이것을 어떻게 설계하느냐에 따라 에이전트의 성격이 결정되며, 프롬프트 엔지니어링의 핵심.

```
model = genai.GenerativeModel(  
    model_name="gemini-2.5-flash",  
    system_instruction="당신은 친절하고 여행 전문가입니다. 항상 구체적인 여행 정보를 제공하세요."  
)
```

Gemini API 대화 구조

LLM API 연동 및 요청·응답 구조 이해

- 단순히 `generate_content()`를 반복 호출하면 이전 대화를 기억하지 못함.
`start_chat()`을 사용하면 대화 기록이 자동으로 유지됨.

```
# 대화 맥락이 유지되지 않는 방식
response1 = model.generate_content("제주도 여행 코스 추천해줘")
response2 = model.generate_content("숙소 추천도 해줄 수 있어?")
# → 두 번째 질문에서 앞의 대화를 기억하지 못함

# 대화 맥락이 유지되는 방식
chat = model.start_chat(history=[])

response1 = chat.send_message("제주도 여행 코스 추천해줘")
print(response1.text)

response2 = chat.send_message("숙소 추천도 해줄 수 있어?")
print(response2.text)
# → "제주도"라는 맥락을 유지하며 숙소를 추천
```

- `history`에는 이전 대화가 자동으로 쌓이며, 대화가 길어질수록 토큰 사용량도 함께 증가함.
AI 에이전트의 Memory(기억) 구성요소가 실제로 이 방식으로 구현됨.

Gemini API 응답 구조

LLM API 연동 및 요청·응답 구조 이해

- API 응답에서 필요한 데이터를 추출하는 방법.

```
response = model.generate_content("안녕하세요!")

# 응답 텍스트 추출
print(response.text)

# 토큰 사용량 확인
print(f"입력 토큰: {response.usage_metadata.prompt_token_count}")
print(f"출력 토큰: {response.usage_metadata.candidates_token_count}")
print(f"전체 토큰: {response.usage_metadata.total_token_count}")
```

```
# Gemini 라이브러리 설치
pip install google-generativeai
```

- 토큰 사용량을 확인하는 습관이 중요.

무료 티어의 일일 요청 한도를 효율적으로 관리하는 데 도움이 됨.

OpenAI API 요청 구조 (참고)

LLM API 연동 및 요청·응답 구조 이해

- **LangChain 실습에서 사용할 OpenAI API 구조를 미리 참고용으로 안내.**

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "당신은 친절한 AI 어시스턴트입니다."},
        {"role": "user", "content": "안녕하세요!"}
    ],
    max_tokens=500,
    temperature=0.7
)

print(response.choices[0].message.content)
```

- Gemini API와 구조가 유사하지만 세부 문법이 다름. 차후에 자세히 다룰 예정.

실습: 나만의 AI 어시스턴트 만들기

함수

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💡 프롬프트 예시

Python으로 Gemini API를 활용한 간단한 AI 어시스턴트를 만들어줘. 조건은 다음과 같아.

- .env 파일에서 Gemini API Key를 불러오도록 구현
- system_instruction으로 AI의 역할을 자유롭게 설정 (예: 여행 전문가, 요리사, 공부 도우미 등)
- while True 루프로 대화가 계속 이어지도록 구현
- 대화 기록이 유지되도록 chat 세션 활용
- "종료" 입력 시 프로그램 종료
- API 호출 실패 시 예외처리 포함
- 모델은 gemini-2.5-flash 사용

🔍 확인해 볼 것

- system_instruction을 바꿔가며 AI의 성격이 어떻게 달라지는지 확인
- 대화가 길어질수록 이전 맥락을 유지하며 응답하는지 확인

AI 에이전트 설계 사고



우리가 만든 것

AI 에이전트 설계 사고

- **앞차시에서 Gemini API를 연동해 만든 간단한 어시스턴트는 진짜 AI 에이전트일까?**

```
chat = model.start_chat(history=[])

while True:
    user_input = input("나: ")
    if user_input == "종료":
        break
    response = chat.send_message(user_input)
    print(f"AI: {response.text}")
```

- **이 코드의 한계:**
 - 대화는 가능하지만 스스로 행동하지 못함
 - 외부 정보(날씨, 뉴스 등)를 가져오지 못함
 - 복잡한 작업을 단계별로 처리하지 못함
 - 목표를 스스로 설정하고 실행하지 못함
- 단순 챗봇과 AI 에이전트의 차이가 바로 여기서 발생.

챗봇 vs AI 에이전트

AI 에이전트 설계 사고

- **차이를 짚어보기**
 - 앞서 만든 AI 어시스턴트는 사용자가 질문하면 답변하는 구조. 대화 맥락은 유지되지만 스스로 무언가를 하지는 않음. 사용자가 "오늘 날씨 어때?"라고 물으면 학습된 데이터를 바탕으로 답할 뿐, 실제 날씨 API를 호출해서 현재 날씨를 가져오지는 못함. 사용자가 "뉴스 요약해줘"라고 하면 뉴스를 직접 검색하지 않고 알고 있는 정보만 답함.
 - AI 에이전트는 여기서 한 단계 더 나아감. 질문을 받으면 스스로 필요한 도구를 선택하고, 정보를 수집하고, 단계별로 작업을 처리한 뒤 최종 결과를 돌려줌.
즉, 단순히 답변하는 것이 아니라 목표를 향해 스스로 행동하는 것이 핵심 차이.
 - 이 차이를 만드는 네 가지 구성요소(LLM, Tool, Memory, Workflow)를 설계 관점에서 구체적으로 살펴봄.

	챗봇	AI 에이전트
동작 방식	질문 → 답변 (수동적)	목표 설정 → 계획 → 실행 (능동적)
외부 도구	사용 불가	검색, API, 코드 실행 등 활용
기억	대화 기록만 유지	장기 기억 저장 및 검색 가능
작업 처리	단일 응답	다단계 작업 처리 가능

AI 에이전트 설계의 핵심 질문

AI 에이전트 설계 사고

- **AI 에이전트를 만들기 전에 반드시 답해야 할 설계 질문들.**
 - 무엇을 하는 에이전트인가? → 목표와 역할 정의
 - 어떤 정보가 필요한가? → 필요한 Tool 파악
 - 무엇을 기억해야 하는가? → Memory 설계
 - 어떤 순서로 작동하는가? → Workflow 설계
- **좋은 에이전트는 코딩보다 설계가 먼저.**
구성요소를 명확히 정의할수록 구현이 쉬워지고 오류가 줄어듦.

LLM: 에이전트의 두뇌

AI 에이전트 설계 사고

- LLM은 에이전트의 모든 판단을 담당.
사용자의 요청을 이해하고, 어떤 Tool을 써야 할지 결정하고, 최종 응답을 생성하는 역할.
- LLM 설계 시 고려할 것:
 - 어떤 모델을 쓸 것인가? → 작업 난이도와 비용 고려
 - system_instruction을 어떻게 설계할 것인가? → 에이전트의 역할과 제약 조건 정의
 - 응답 형식을 어떻게 제어할 것인가? → JSON, 마크다운 등 출력 형식 지정

system_instruction이 에이전트의 역할을 결정

```
model = genai.GenerativeModel(  
    model_name="gemini-2.5-flash",  
    system_instruction="""  
당신은 뉴스 요약 전문 에이전트입니다.  
- 항상 핵심 내용을 3줄 이내로 요약하세요.  
- 출처를 반드시 명시하세요.  
- 주관적인 의견을 포함하지 마세요.  
""")
```

Tool: 에이전트의 손발

AI 에이전트 설계 사고

- Tool은 LLM이 스스로 할 수 없는 실제 행동을 대신 수행.
Python 함수로 구현되며, 에이전트가 필요에 따라 호출.

Tool 종류	예시	활용
검색 Tool	Google Search API, Tavily	실시간 정보 검색
날씨 Tool	OpenWeather API	날씨 정보 조회
계산 Tool	Python 함수	수식 계산, 데이터 처리
파일 Tool	파일 읽기/쓰기 함수	문서 처리
데이터베이스 Tool	DB 조회 함수	저장된 데이터 검색

```
# Tool은 Python 함수로 구현
def search_news(keyword: str) -> str:
    """뉴스를 검색하는 Tool"""
    # 실제 검색 API 호출
    return f"{keyword} 관련 최신 뉴스 결과"

def summarize(text: str) -> str:
    """텍스트를 요약하는 Tool"""
    return f"{text[:100]}... (요약됨)"
```

Memory: 에이전트의 기억

AI 에이전트 설계 사고

- **Memory**는 에이전트가 이전 대화와 작업 결과를 저장하고 필요할 때 꺼내 쓸 수 있게 하는 구성요소.

Memory 종류	설명	구현 방식
단기 기억	현재 대화 맥락 유지	chat.history
장기 기억	이전 세션 정보 저장	데이터베이스, 파일 저장
벡터 메모리	의미 기반 정보 검색	벡터 데이터베이스 (RAG에서 활용)

```
# 단기 기억만 있는 경우
# 대화 종료 후 모든 정보가 사라짐
```

```
# 장기 기억을 추가한 경우
import json
```

```
def save_memory(data: dict, filename="memory.json"):
    with open(filename, "w") as f:
        json.dump(data, f)
```

```
def load_memory(filename="memory.json"):
    with open(filename, "r") as f:
        return json.load(f)
```

Workflow: 에이전트의 실행 계획

AI 에이전트 설계 사고

- **Workflow**는 에이전트가 목표를 달성하기 위해 작업을 어떤 순서로 처리할지 설계하는 구성요소.

```
# Workflow 없이 — 모든 걸 한 번에 처리
response = model.generate_content(user_input)

# Workflow 있는 경우 — 단계별로 처리
def run_news_agent(user_input):
    # 1단계: 키워드 추출
    keyword = extract_keyword(user_input)

    # 2단계: 뉴스 검색
    news = search_news(keyword)

    # 3단계: 요약
    summary = summarize(news)

    # 4단계: 최종 응답 생성
    response = generate_response(summary)

    return response
```

- 단계가 명확할수록 디버깅이 쉬워지고 각 단계를 독립적으로 개선할 수 있음.
LangGraph에서 이 Workflow를 시각적으로 설계하는 방법을 다룰 예정.

네 가지 요소가 함께 작동하는 구조

AI 에이전트 설계 사고

- AI 에이전트는 네 가지 요소가 유기적으로 연결될 때 비로소 완성됨.
아래 코드는 단순해 보이지만, 이 구조가 앞으로 만들 모든 에이전트의 기반이 됨.

```
def news_agent(user_input):  
    # LLM: 사용자 의도 파악  
    keyword = extract_keyword(user_input)    # LLM 호출  
  
    # Memory: 이전 검색 기록 확인  
    history = load_memory()                  # Memory 참조  
  
    # Tool: 실제 뉴스 검색 실행  
    news = search_news(keyword)              # Tool 실행  
  
    # Workflow: 검색 → 요약 → 응답 순서로 처리  
    summary = summarize(news)                # Tool 실행  
    response = generate_response(summary)    # LLM 호출  
  
    # Memory: 결과 저장  
    save_memory({"keyword": keyword, "summary": summary})  
  
    return response
```

실습: 에이전트 설계해보기

AI 에이전트 설계 사고

- 코딩 전에 에이전트를 먼저 설계해보는 연습.
만들고 싶은 에이전트를 하나 골라 아래 양식을 채워보세요.
- 에이전트 이름: _____
에이전트 목표: _____

구성요소	설명	내용
LLM	어떤 모델? system_instruction은 어떻게 설정할 것인가?	
Tool	어떤 기능(함수)이 필요한가?	
Memory	무엇을 기억해야 하는가?	
Workflow	어떤 순서로 작동하는가?	

에이전트 설계 예시

AI 에이전트 설계 사고

- 코딩 전에 에이전트를 먼저 설계해보는 연습.
만들고 싶은 에이전트를 하나 골라 아래 양식을 채워보세요.
- 에이전트 이름: 뉴스 검색·요약 에이전트
에이전트 목표: 사용자가 입력한 키워드로 최신 뉴스를 검색하고 핵심 내용을 요약해서 전달

구성요소	설명	내용
LLM	어떤 모델? system_instruction은 어떻게 설정할 것인가?	gemini-2.5-flash / "당신은 뉴스 요약 전문가입니다. 핵심 내용을 3줄 이내로 요약하세요."
Tool	어떤 기능(함수)이 필요한가?	뉴스 검색 함수, 텍스트 요약 함수
Memory	무엇을 기억해야 하는가?	이전에 검색한 키워드와 요약 결과 저장
Workflow	어떤 순서로 작동하는가?	키워드 추출 → 뉴스 검색 → 요약 → 응답 출력

MBTI 기반 콘텐츠 추천 에이전트



MBTI 기반 콘텐츠 추천 에이전트

MBTI 기반 콘텐츠 추천 에이전트

- **MBTI 유형을 입력하면 그에 맞는 콘텐츠(영화, 책, 음악 등)를 추천해주는 에이전트를 만들기.**
- **이 실습에서 다루는 핵심 개념:**
 - Gemini API 연동
 - system_instruction으로 에이전트 역할 설정
 - 사용자 입력에 따라 맞춤형 응답 생성
 - 프롬프트 엔지니어링 기초
- **완성 예시:**

사용자: MBTI가 INFP인데 볼만한 영화 추천해줘

에이전트: INFP 유형에게 어울리는 영화를 추천해드릴게요!

1. 비포 선라이즈 — 감성적이고 철학적인 대화가 INFP의 내면세계와 잘 맞아요.
2. 리틀 포레스트 — 자연 속에서 자신을 찾아가는 이야기...

에이전트 설계

MBTI 기반 콘텐츠 추천 에이전트

- 코딩 전에 구조를 먼저 설계.
 - 에이전트 이름: MBTI 콘텐츠 추천 에이전트
- 에이전트 목표: 사용자의 MBTI 유형에 맞는 콘텐츠를 추천

구성요소	내용
LLM	gemini-2.5-flash / MBTI 전문가이자 콘텐츠 큐레이터 역할
Tool	이번 실습에서는 LLM 자체 지식 활용 (외부 API 없음)
Memory	대화 기록 유지 — 이전 추천 내용을 참고해 이어서 대화 가능
Workflow	MBTI 입력 → 콘텐츠 유형 선택 → 맞춤 추천 → 응답 출력

프롬프트 엔지니어링: system_instruction 설계

MBTI 기반 콘텐츠 추천 에이전트

- system_instruction을 어떻게 설계하느냐에 따라 추천 품질이 달라짐.

```
# 너무 단순한 system_instruction
system_instruction = "MBTI에 맞는 콘텐츠를 추천해줘"
```

```
# 구체적으로 역할과 방식을 정의한 system_instruction
system_instruction = """
당신은 MBTI 전문가이자 콘텐츠 큐레이터입니다.
사용자의 MBTI 유형에 맞는 영화, 책, 음악을 추천해주세요.
```

추천 시 다음 원칙을 따르세요:

- 각 MBTI 유형의 성격 특성을 반영한 추천
- 추천 이유를 MBTI 특성과 연결해서 설명
- 친근하고 공감 가는 말투 사용
- 콘텐츠별로 최소 2개 이상 추천

```
"""
```

- system_instruction이 구체적일수록 에이전트의 응답 품질이 높아짐.
직접 바꿔가며 차이를 확인해보기.

실습: 에이전트 만들기

MBTI 기반 콘텐츠 추천 에이전트

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💡 프롬프트 예시

Python으로 Gemini API를 활용한 MBTI 기반 콘텐츠 추천 에이전트를 만들어줘. 조건은 다음과 같아.

- .env 파일에서 Gemini API Key를 불러오도록 구현
- system_instruction은 아래 내용을 그대로 사용:
당신은 MBTI 전문가이자 콘텐츠 큐레이터입니다. 사용자의 MBTI 유형에 맞는 영화, 책, 음악을 추천해주세요.
추천 시 다음 원칙을 따르세요:
 - 각 MBTI 유형의 성격 특성을 반영한 추천
 - 추천 이유를 MBTI 특성과 연결해서 설명
 - 친근하고 공감 가는 말투 사용
 - 콘텐츠별로 최소 2개 이상 추천
- 사용자가 MBTI 유형과 원하는 콘텐츠 유형(영화/책/음악)을 입력하면 맞춤 추천
- 대화 기록이 유지되도록 chat 세션 활용
- while True 루프로 대화가 계속 이어지도록 구현
- "종료" 입력 시 프로그램 종료
- 모델은 gemini-2.5-flash 사용

더 발전시켜보기

MBTI 기반 콘텐츠 추천 에이전트

- **완성한 에이전트에 아래 기능 중 하나를 골라 직접 추가해보세요.**
앞에서 했던 것처럼 AI에게 추가 요청을 넣어서 코드를 발전시켜 나가면 됩니다.

추가 기능	프롬프트 힌트
MBTI 유형 유효성 검사	"잘못된 MBTI 입력 시 다시 입력받도록 수정해줘"
콘텐츠 유형 선택 메뉴 추가	"영화/책/음악 중 선택하는 메뉴를 추가해줘"
추천 결과를 파일로 저장	"추천 결과를 txt 파일로 저장하는 기능 추가해줘"
멀티 MBTI 비교	"두 가지 MBTI를 입력하면 궁합과 함께 두 유형이 공통으로 즐길 수 있는 콘텐츠를 추천하는 기능 추가해줘"

뉴스 검색·요약 에이전트



뉴스 검색·요약 에이전트

뉴스 검색·요약 에이전트

- 사용자가 입력한 키워드로 실시간 뉴스를 검색하고 핵심 내용을 요약해주는 에이전트 만들기
- **MBTI 에이전트와의 차이점:**
 - LLM 자체 지식이 아닌 외부 Tool(뉴스 검색 API)을 실제로 연동
 - Tool을 Python 함수로 직접 구현해보는 첫 번째 실습
 - 에이전트 구성요소 중 Tool이 실제로 어떻게 작동하는지 체험
- **완성예시**

사용자: AI 관련 최신 뉴스 검색해줘

에이전트: AI 관련 최신 뉴스를 검색했습니다.

1. [제목] — 핵심 내용 2~3문장 요약...
2. [제목] — 핵심 내용 2~3문장 요약...
3. [제목] — 핵심 내용 2~3문장 요약...

에이전트 설계

뉴스 검색·요약 에이전트

- **에이전트 이름:** 뉴스 검색·요약 에이전트
에이전트 목표: 사용자가 입력한 키워드로 최신 뉴스를 검색하고 핵심 내용을 요약해서 전달

구성요소	내용
LLM	gemini-2.5-flash / 뉴스 요약 전문가 역할, 객관적이고 간결한 요약
Tool	뉴스 검색 함수 (NewsData API 활용)
Memory	이전 검색 키워드와 결과 유지
Workflow	키워드 입력 → 뉴스 검색(Tool) → 요약(LLM) → 응답 출력

NewsData API Key 발급

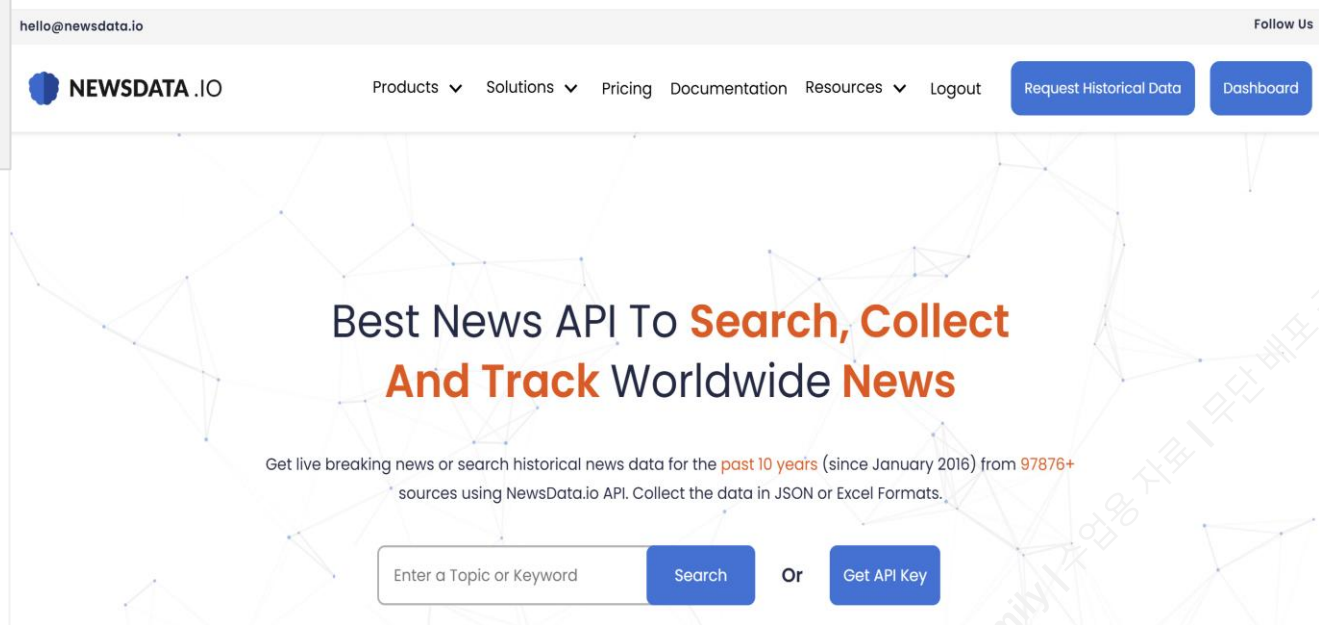
뉴스 검색·요약 에이전트

- NewsData API Key 발급 방법.

- 발급 순서:
 - newsdata.io 접속 및 회원가입
 - 대시보드에서 API Key 확인
 - 생성된 키 복사 후 .env 파일에 저장

.env 파일에 추가

```
NEWSDATA_API_KEY=pub_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
```



뉴스 검색 Tool 만들기

뉴스 검색·요약 에이전트

- **news_tool.py** 파일로 저장하고 실행해보세요.

```
import requests
import os
from dotenv import load_dotenv

load_dotenv()
def search_news(keyword: str) -> str:
    """키워드로 뉴스를 검색하는 Tool"""
    url = "https://newsdata.io/api/1/news"
    params = {
        "apikey": os.getenv("NEWSDATA_API_KEY"),
        "q": keyword,
        "language": "ko",
        "size": 5
    }
    response = requests.get(url, params=params)
    articles = response.json().get("results", [])
    result = ""
    for article in articles:
        result += f"제목: {article['title']}\n"
        result += f"내용: {article.get('description', '내용 없음')}\n\n"
    return result if result else "관련 뉴스를 찾을 수 없습니다."
```

함수 실행 테스트

```
print(search_news("인공지능"))
```

프롬프트 엔지니어링: 요약 품질 높이기

뉴스 검색·요약 에이전트

- **news_tool.py** 파일로 저장하고 실행해보세요.

```
import requests
import os
from dotenv import load_dotenv

load_dotenv()
def search_news(keyword: str) -> str:
    """키워드로 뉴스를 검색하는 Tool"""
    url = "https://newsdata.io/api/1/news"
    params = {
        "apikey": os.getenv("NEWSDATA_API_KEY"),
        "q": keyword,
        "language": "ko",
        "size": 5
    }
    response = requests.get(url, params=params)
    articles = response.json().get("results", [])
    result = ""
    for article in articles:
        result += f"제목: {article['title']}\n\n"
        result += f"내용: {article.get('description', '내용 없음')}\n\n"
    return result if result else "관련 뉴스를 찾을 수 없습니다."
```

함수 실행 테스트

```
print(search_news("인공지능"))
```

프롬프트 엔지니어링: 요약 품질 높이기

뉴스 검색·요약 에이전트

- 뉴스 요약 에이전트에서 system_instruction은 요약의 품질과 형식을 결정.

너무 단순한 system_instruction

```
system_instruction = "뉴스를 요약해줘"
```

구체적으로 역할과 형식을 정의한 system_instruction

```
system_instruction = """
```

당신은 뉴스 요약 전문가입니다.

검색된 뉴스를 바탕으로 핵심 내용을 요약해주세요.

요약 시 다음 원칙을 따르세요:

- 각 뉴스의 핵심 내용을 2~3문장으로 요약
- 중립적이고 객관적인 표현 사용
- 중요도 순으로 정렬해서 제시
- 출처(뉴스 제목)를 함께 표시

```
"""
```

실습: 에이전트 만들기

뉴스 검색·요약 에이전트

- 아래 프롬프트를 평소 사용하는 AI(ChatGPT, Claude 등)에 입력하고, 생성된 코드를 VSCode에서 실행해보세요.

💡 프롬프트 예시

Python으로 Gemini API와 NewsData API를 활용한 뉴스 검색·요약 에이전트를 만들어줘. 조건은 다음과 같아.

- .env 파일에서 GEMINI_API_KEY와 NEWSDATA_API_KEY를 불러오도록 구현
- search_news(keyword) 함수로 NewsData API에서 한국어 뉴스를 검색하고 제목과 내용을 반환
- system_instruction은 아래 내용을 그대로 사용:
 - 당신은 뉴스 요약 전문가입니다.
 - 검색된 뉴스를 바탕으로 핵심 내용을 요약해주세요.
 - 각 뉴스의 핵심 내용을 2~3문장으로 요약
 - 중립적이고 객관적인 표현 사용
 - 중요도 순으로 정렬해서 제시
 - 출처(뉴스 제목)를 함께 표시
- 사용자가 키워드를 입력하면 search_news()로 뉴스를 검색하고 LLM이 요약해서 응답
- 대화 기록이 유지되도록 chat 세션 활용
- while True 루프로 대화가 계속 이어지도록 구현
- "종료" 입력 시 프로그램 종료
- API 호출 실패 시 예외처리 포함
- 모델은 gemini-2.5-flash 사용

더 발전시켜보기

뉴스 검색·요약 에이전트

- **완성한 에이전트에 아래 기능 중 하나를 골라 직접 추가해보세요.**
앞에서 했던 것처럼 AI에게 추가 요청을 넣어서 코드를 발전시켜 나가면 됩니다.

추가 기능	프롬프트 힌트
검색 결과 개수 조절	"사용자가 검색할 뉴스 개수를 입력할 수 있도록 수정해줘"
카테고리 선택 기능 추가	"정치/경제/IT/스포츠 카테고리 중 선택할 수 있도록 수정해줘"
검색 기록 저장	"검색한 키워드와 요약 결과를 txt 파일로 저장하는 기능 추가해줘"
날짜 범위 뉴스 검색	"사용자가 검색 날짜 범위를 입력하면 해당 기간의 뉴스만 검색하도록 수정해줘"

오늘 하루도 수고하셨습니다